

Persistence in LangGraph

How LangGraph saves and restores workflow state over time — checkpoints, threads, fault tolerance, and time travel.

WHAT THIS CHAPTER COVERS

- ▶ What persistence means in LangGraph
- ▶ Why intermediate state values matter
- ▶ Checkpointers and superstep-based checkpoints
- ▶ Threads and thread_id isolation
- ▶ Implementing persistence with InMemorySaver
- ▶ Fault tolerance — resuming after a crash
- ▶ Time travel — replaying and forking execution
- ▶ Updating state at a checkpoint

Persistence in LangGraph

RECAP — CHAPTER 9

Chapter 9 built a single-node chatbot and showed that, by default, a workflow's state is scoped only to one `invoke()` call and is erased the moment execution reaches **END**. It introduced **MemorySaver** and `thread_id` just enough to make chat memory work. This chapter formalizes that mechanism as **Persistence** — the foundational LangGraph concept that many later features (fault tolerance, human-in-the-loop, time travel) are built on top of.

1. What Is Persistence?

Definition

Persistence in LangGraph refers to the ability to **save and restore the state of a workflow over time**.

Why It Exists — Recap of the Two Core LangGraph Ideas

Understanding persistence requires recalling the two foundational principles already covered:

1. **Concept of Graph** — any high-level goal can be decomposed into a set of tasks, represented as a graph where each *node* is a task and each *edge* represents execution order.
2. **Concept of State** — a graph needs important pieces of data to execute (e.g. the messages exchanged in a chatbot). This data is stored in the *State*, a dictionary that every node can read from and write to.

The Problem It Solves

Using graphs and state, arbitrarily complex LLM-based workflows can be built and executed. But a specific behavior emerges during execution: when `invoke()` triggers a workflow, input flows from **START** through each node, and the state changes at every node. Once execution finishes, LangGraph's fundamental default behavior is that **all state values are erased** — they disappear from RAM, and cannot be recovered afterward, even if they are needed in the future.

Persistence changes this core behavior. With persistence, the state of a workflow can be **saved somewhere** (typically a database) so that it can be inspected and reused at any future point.

🔄 Quick Revision — What Is Persistence

- Persistence = the ability to save and restore a workflow's state over time.
- Without persistence, all state is erased from RAM the moment a workflow reaches **END**.
- Persistence builds directly on the Graph and State concepts already established.
- Persistence is considered a foundational LangGraph topic — many later features are built on it.

2. Speciality of Persistence

Not Just the Final Value — Every Intermediate Value

The defining feature of persistence is that it does not store only the **final** state value — it stores **every intermediate value** the state passes through, at every stage of execution.

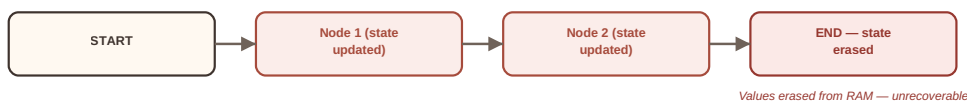
Example

Consider a two-node workflow with a state variable `name`:

- At START, `name` = "A".
- Node 1 changes it to `name` = "B".
- Node 2 changes it to `name` = "C".
- The workflow ends with the final value `name` = "C".

With persistence enabled, LangGraph does not save only "C". It saves the value of `name` at START, after Node 1, after Node 2, and at END — four separate stored snapshots for this one execution.

WITHOUT PERSISTENCE (default LangGraph behavior)



WITH PERSISTENCE (checkpointer attached)



Figure 10.1 — Without persistence, state is erased at END; with persistence, every intermediate and final value is saved to a database.

IMPORTANT

Persistence is not simply "final-value caching." Its defining property is capturing **every intermediate stage** of the state, which is precisely what makes fault tolerance and time travel possible later in this chapter.

Quick Revision — Speciality of Persistence

- Persistence stores the state at every intermediate stage, not just the final value.
- Each stage's snapshot is saved independently as the workflow progresses.
- This intermediate-value storage is the mechanism that enables both fault tolerance and time travel.

3. The Checkpointer

Definition

Persistence in LangGraph is implemented via a **Checkpointer**. A checkpointer divides the entire execution of a graph into *checkpoints*, and at every checkpoint it saves the current state values to a database.

How Checkpoints Are Decided

Every **superstep** of a graph becomes one checkpoint. Recall that when multiple nodes execute in parallel (e.g. three branches running simultaneously), all of those parallel executions together count as a single superstep. So a graph with a linear `START → Node 1 → Node 2 → Node 3 → END` path has one checkpoint at each of those five points.

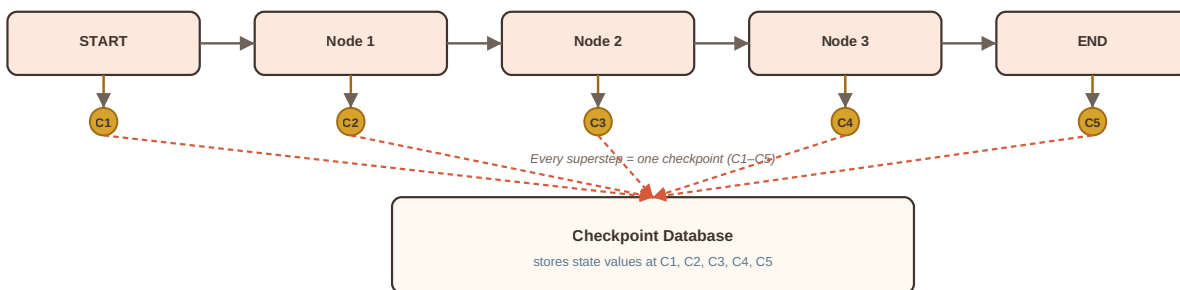


Figure 10.2 — Every superstep becomes a checkpoint (C1–C5); the checkpointer saves state values into the database at each one.

WORKED EXAMPLE

Suppose the state has a `numbers` attribute (a list of integers) with a reducer that merges new values instead of replacing them:

- At the `START` checkpoint, `numbers = [1]` — saved.
- After `Node 1` produces 2, `numbers = [1, 2]` — saved (merged via the reducer, not replaced).
- After three parallel nodes produce 3, 4, 5, `numbers = [1, 2, 3, 4, 5]` — saved.
- At `END`, the value is unchanged, `[1, 2, 3, 4, 5]` — saved.

Four checkpoints in this graph produce four stored state snapshots in the database.

BEST PRACTICE

`InMemorySaver` (imported from `langgraph.checkpoint.memory`) is a simple checkpointer that stores everything in RAM. It is ideal for demos and learning, but is **not used in production** since values vanish when the process stops. Production setups use durable checkpointers such as the Postgres or Redis checkpointers.

🔄 Quick Revision — The Checkpointer

- Checkpointer = the component that implements persistence by dividing execution into checkpoints.
- Every superstep of the graph corresponds to exactly one checkpoint.
- At each checkpoint, the checkpointer saves the current state values to a database.
- InMemorySaver stores checkpoints in RAM — useful for demos, not for production.
- Production checkpointer (e.g. Postgres, Redis) persist durably across restarts.

4. Threads

The Problem It Solves

If every execution of a workflow saves its checkpoints to the same database, how does LangGraph know which stored values belong to which particular execution when there are many? This is the problem **threads** solve.

How It Works

Every time a workflow is executed with persistence enabled, a **unique thread_id** is assigned via the config passed to `invoke()`. All checkpoint values generated during that execution are stored in the database **against that thread_id**. A different `invoke()` call with a different `thread_id` keeps its checkpoints completely isolated from the first.

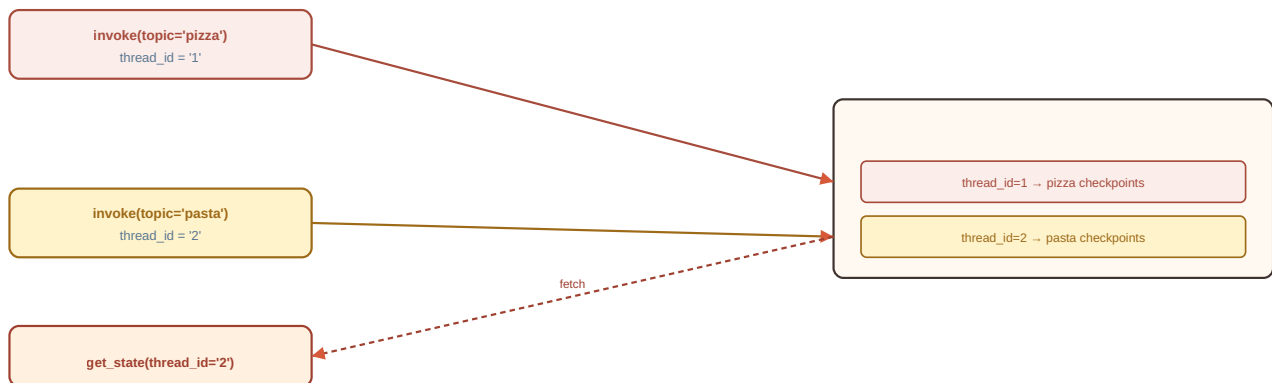


Figure 10.3 — Two `invoke()` calls with different `thread_ids` keep their checkpoint histories isolated in the database.

CHATBOT USE CASE

When a user starts a new conversation, a new `thread_id` is created and every exchange is stored against it. Two days later, when the same user wants to resume that specific conversation, the app fetches the corresponding `thread_id` and retrieves all messages stored against it — automatically restoring the full chat history.

INTERVIEW TIP

If asked to distinguish checkpointer vs. thread: the **checkpointer** is the mechanism that saves state at each checkpoint; the **thread_id** is the key used to group and isolate the checkpoints belonging to one particular execution/session.

Quick Revision — Threads

- A `thread_id` uniquely tags one particular workflow execution/session.
- All checkpoint values from that execution are stored against its `thread_id` in the database.
- Different executions (different `thread_ids`) remain fully isolated in the same database.
- `get_state(config)` with a given `thread_id` retrieves only that execution's state.
- Threads are what make resumable, multi-session chatbots possible.

5. Implementing Persistence — Joke Generator Workflow

The Workflow

A simple two-node sequential workflow: given a topic, `joke_generator` generates a joke about it, and `joke_explainer` generates an explanation of that joke.

Setup and State

```
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph import StateGraph, START, END
from typing import TypedDict
from langchain_groq import ChatGroq
from dotenv import load_dotenv

load_dotenv()
model = ChatGroq(model = "openai/gpt-oss-20b")

class JokeState(TypedDict):
    topic : str
    joke : str
    explanation : str
```

Node Functions

```
def generate_joke(state : JokeState):
    topic = state['topic']
    prompt = f"Give me a funny joke on the topic : {topic}."
    joke = model.invoke(prompt).content
    return {'joke':joke}

def explain_joke(state : JokeState):
    joke = state['joke']
    prompt = f"Explain this joke to me in a simple way : {joke}."
    explanation = model.invoke(prompt).content
    return {'explanation':explanation}
```

Graph Assembly

```
graph = StateGraph(JokeState)

graph.add_node("joke_generator", generate_joke)
graph.add_node("joke_explainer", explain_joke)

graph.add_edge(START, "joke_generator")
graph.add_edge("joke_generator", "joke_explainer")
graph.add_edge('joke_explainer', END)
```

Attaching the Checkpointer and Invoking

A checkpointer object (`InMemorySaver`) is created and passed to `compile()`. This single line tells LangGraph to save every intermediate plus final state value to memory. Invocation additionally requires a `config` dict carrying a `thread_id`:

```
checkpointer = InMemorySaver()

workflow = graph.compile(checkpointer = checkpointer)

config1 = {'configurable':{'thread_id':'1'}}

workflow.invoke({'topic':'cold drink'}, config = config1)
```

Retrieving State

Two retrieval calls matter here:

- `workflow.get_state(config1)` — returns only the **final** `StateSnapshot` for the given thread.
- `list(workflow.get_state_history(config1))` — returns **every** stored `StateSnapshot` for the thread, most recent first, including intermediate ones.

```

workflow.get_state(config1)
# StateSnapshot(values={'topic': 'cold drink', 'joke': '...', 'explanation': '...'},
next=(), ...)

list(workflow.get_state_history(config1))
# 4 snapshots: before START, before joke_generator, before joke_explainer, final (after
joke_explainer)

```

For this two-node graph, `get_state_history` returns exactly four snapshots — one per checkpoint: the empty pre-START state (`next=('__start__',)`), the state after START holding only `topic` (`next=('joke_generator',)`), the state after joke generation holding `topic` + `joke` (`next=('joke_explainer',)`), and the final state holding all three fields (`next=()`).

IMPORTANT

Each `StateSnapshot`'s `next` field tells you which node will execute next from that checkpoint — this is exactly the information `LangGraph` uses internally to know where to resume from.

Running with a Different Thread

Invoking again with `topic='pasta'` and a new `config = {'configurable': {'thread_id': '2'}}` produces a completely separate, isolated set of checkpoints. `get_state/get_state_history` called with `config1` (thread 1) still returns only the cold-drink data; calling with the thread-2 config returns only the pasta data. Both are retrievable at any later point since both are stored in the database against their own `thread_id`.

🔄 Quick Revision — Implementing Persistence

- `InMemorySaver` is imported from `langgraph.checkpoint.memory`.
- Pass `checkpointer=checkpointer` to `graph.compile()` to enable persistence.
- `invoke()` requires a config dict with a `configurable.thread_id` key once a checkpointer is attached.
- `get_state(config)` returns only the final `StateSnapshot` for that thread.
- `get_state_history(config)` returns every stored snapshot for that thread, most recent first.
- Each `StateSnapshot`'s `next` field indicates which node would execute next from that point.

6. Benefits of Persistence

Persistence unlocks four major capabilities in `LangGraph`:

Benefit	What It Enables
Short-Term Memory	Chatbots can resume a past conversation by reloading messages stored against a <code>thread_id</code> .
Fault Tolerance	A crashed workflow can resume exactly where it stopped, instead of restarting from START.

Human-in-the-Loop (HITL)	Execution can be interrupted, wait indefinitely for human input, and later resume from the exact interrupt point.
Time Travel	Execution can be replayed from any past checkpoint, optionally with modified state, creating a new branch.

1. SHORT-TERM MEMORY

Chat tools like ChatGPT offer "start a new conversation" or "resume an old conversation." Resuming is only possible because past messages were stored via persistence, keyed by `thread_id`. Persistence is the only way to implement short-term memory of this kind in LangGraph.

2. FAULT TOLERANCE

Covered in depth in Section 7 below.

3. HUMAN-IN-THE-LOOP

Consider a workflow that generates a LinkedIn post and then, before publishing it via the LinkedIn API, pauses to ask a human for approval. The human's response might arrive instantly, after an hour, or after two days — the workflow cannot stay active in memory for that entire window. LangGraph solves this by **interrupting** (temporarily suspending) execution at that point and waiting. Because every checkpoint prior to the interrupt was already saved, the workflow knows exactly how to resume once the human's input arrives. This is conceptually similar to fault tolerance, except the "pause" is intentional rather than caused by an external failure.

REAL WORLD INSIGHT

Human-in-the-loop is deliberately not implemented with code in this chapter — it is covered in a dedicated later video/chapter, where persistence is used again as the enabling mechanism.

4. TIME TRAVEL

Covered in depth in Section 8 below.

🔄 Quick Revision — Benefits of Persistence

- Persistence enables four capabilities: short-term memory, fault tolerance, human-in-the-loop, and time travel.
- Short-term memory: resumable chat, made possible by messages stored against a `thread_id`.
- Human-in-the-loop pauses (interrupts) a workflow indefinitely and resumes it once human input arrives.
- HITL and fault tolerance both rely on resuming from a saved checkpoint — HITL's pause is intentional, fault tolerance's is unplanned.

7. Fault Tolerance in Practice

Scenario

A three-step workflow is built where Step 2 deliberately hangs (simulating a long-running or crashed operation). While Step 2 is running, execution is manually interrupted (simulating a real crash — e.g. a server going down or an API call failing mid-node).

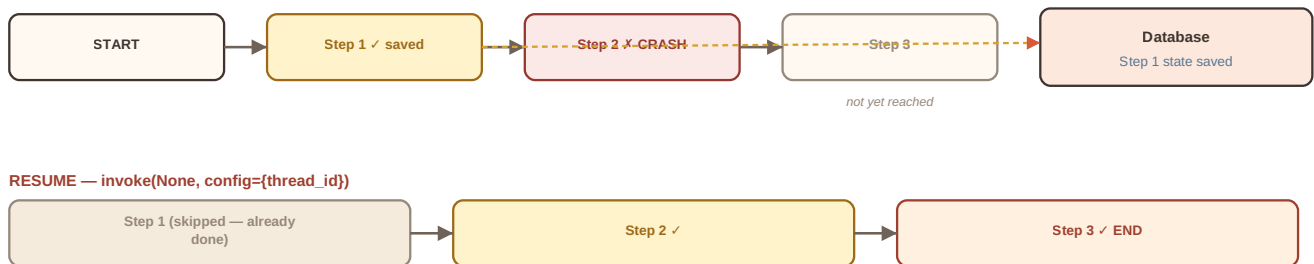


Figure 10.4 — A crash mid-workflow is resumed exactly from the last saved checkpoint, not from *START*, via `invoke(None, config)`.

State and Steps

```
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.memory import InMemorySaver
from typing import TypedDict
import time

class CrashState(TypedDict):
    input: str
    step1: str
    step2: str

def step_1(state: CrashState) -> CrashState:
    print("[OK] Step 1 executed")
    return {"step1": "done", "input": state["input"]}

def step_2(state: CrashState) -> CrashState:
    print("[WAIT] Step 2 hanging...")
    time.sleep(1000) # simulates a long-running / hung node
    return {"step2": "done"}

def step_3(state: CrashState) -> CrashState:
    print("[OK] Step 3 executed")
    return {"done": True}
```

Graph and First (Interrupted) Run

```
builder = StateGraph(CrashState)
builder.add_node("step_1", step_1)
builder.add_node("step_2", step_2)
builder.add_node("step_3", step_3)

builder.set_entry_point("step_1")
builder.add_edge("step_1", "step_2")
builder.add_edge("step_2", "step_3")
builder.add_edge("step_3", END)

checkpointer = InMemorySaver()
graph = builder.compile(checkpointer=checkpointer)

try:
    graph.invoke({"input": "start"}, config={"configurable": {"thread_id": 'thread-1'}})
except KeyboardInterrupt:
    print("[X] Kernel manually interrupted (crash simulated).")
# Output: Step 1 executed, then Step 2 hanging... then manually interrupted
```

Inspecting state at this point (`get_state_history`) shows only three snapshots: the empty pre-START state, the state after START (`input="start"`), and the state after Step 1 (`step1="done"`). Step 2 never completed, so no checkpoint beyond Step 1 exists.

Resuming After the Crash

To resume, `invoke()` is called again with **None** as the input (instead of an initial state) and the **same** `thread_id`. Passing **None** tells LangGraph to resume from wherever this thread last stopped, rather than starting a fresh execution.

```
final_state = graph.invoke(None, config={"configurable": {"thread_id": 'thread-1'}})
print("[OK] Final State:", final_state)
# Execution resumes directly at Step 2 (not Step 1) – Step 1 is not re-run.
# Step 2 completes, then Step 3 executes, then the workflow reaches END.
```

The resumed run's `get_state_history` now shows five snapshots in total, with the newest (final) snapshot's `next` being empty and the oldest showing `next=('step_1',)` — confirming the full chronological chain from START through to completion, resumed mid-way rather than restarted.

COMMON MISTAKE

Passing a fresh initial state dict (e.g. `{"input": "start"}`) instead of **None** when trying to resume will start a **brand-new** execution rather than resuming the interrupted one. Resuming specifically requires **None** as the input, paired with the original thread's `thread_id`.

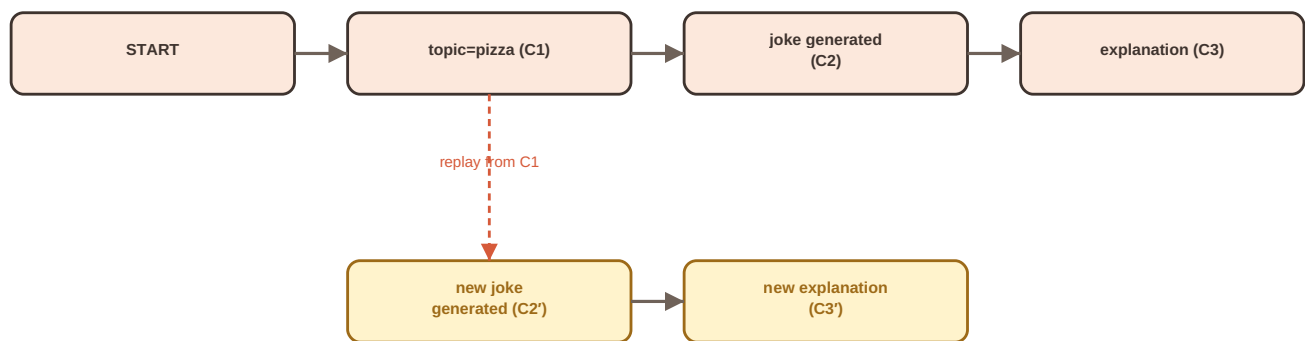
🔄 Quick Revision — Fault Tolerance

- Fault tolerance = resuming a crashed workflow from its last saved checkpoint instead of restarting from START.
- Resume syntax: `graph.invoke(None, config={'configurable': {'thread_id': same_id}})`.
- Passing `None` as input signals 'resume from where this thread last stopped'.
- Only nodes after the last successful checkpoint re-execute; completed nodes are not repeated.
- This benefit exists specifically because persistence saves intermediate — not just final — state.

8. Time Travel

Definition

Time travel is the ability to replay a workflow's execution from any earlier saved checkpoint — and optionally, to modify the state at that checkpoint before replaying. The main practical use case is **debugging** complex workflows: jump back to a specific checkpoint and re-run the downstream logic to reproduce or investigate an issue.



Re-invoking with `checkpoint_id=C1` forks a new branch; the original path (top) remains stored too.

Figure 10.5 — Replaying from an earlier checkpoint (C1) creates a new forked branch; the original execution path remains stored alongside it.

Locating a Checkpoint

Every checkpoint has its own unique **checkpoint_id**, visible in each `StateSnapshot`'s `config` field. To time-travel, first locate the desired snapshot via `get_state_history` and copy its `checkpoint_id`.

```
list(workflow.get_state_history(config1))
# Locate the snapshot where topic='cold drink' but joke has not yet been generated,
# e.g. checkpoint_id = '1f19f13a-5d6c-6fae-8000-3740798a5cd4'

workflow.get_state({'configurable': {'thread_id': '1', 'checkpoint_id':
'1f19f13a-5d6c-6fae-8000-3740798a5cd4'}})
# StateSnapshot(values={'topic': 'cold drink'}, next=('joke_generator',), ...)
```

Replaying From That Checkpoint

Calling `invoke(None, config)` with that specific `checkpoint_id` (alongside the `thread_id`) re-executes everything downstream of that checkpoint:

```
workflow.invoke(None, {'configurable': {'thread_id': '1', 'checkpoint_id':
'1f19f13a-5d6c-6fae-8000-3740798a5cd4'}})
# {'topic': 'cold drink', 'joke': '(new, different joke)', 'explanation': '(new
explanation)'}
```

Because the LLM is probabilistic, the newly generated joke and explanation differ from the originals — even though the topic is identical. `get_state_history` afterward shows the original 4 snapshots *plus* 2 new ones from the replay, since a new branch (`metadata={'source': 'fork', ...}`) was created from the same checkpoint rather than overwriting the original path.

Updating State at a Checkpoint

Time travel can also modify state before replaying, using `update_state()`. This is passed the target checkpoint's config plus a partial state update:

```
workflow.update_state(
    {'configurable': {'thread_id': '1', 'checkpoint_id':
'1f19f13a-5d6c-6fae-8000-3740798a5cd4', 'checkpoint_ns': ''}},
    {'topic': 'samosa'})
# Returns a NEW checkpoint_id representing this updated state, e.g. '...8001-6de4b38f2106'
```

COMMON MISTAKE

After calling `update_state()`, replaying must use the **new** `checkpoint_id` returned by `update_state()` — not the original one. Re-invoking with the old (pre-update) `checkpoint_id` silently regenerates from the un-updated value (e.g. still "pizza" instead of the intended "samosa"), because that checkpoint id still points to the old state snapshot.

```
# Correct: replay from the checkpoint_id returned by update_state()
workflow.invoke(None, {'configurable': {'thread_id': 1, 'checkpoint_id': '1f19f195-
a882-6d84-8001-6de4b38f2106'}})
# {'topic': 'samosa', 'joke': '(joke about samosa)', 'explanation': '(explanation)'}
```

This confirms the update was applied: the new joke and explanation are generated for "samosa", not the original topic. Every fork and update adds new snapshots to `get_state_history`, building a full branching tree of the workflow's execution history for that thread.

🔄 Quick Revision — Time Travel

- Time travel = replaying execution from any earlier checkpoint, primarily used for debugging.
- Every checkpoint has a unique `checkpoint_id`, found in a `StateSnapshot`'s `config` field.
- Replay syntax: `invoke(None, {'configurable': {'thread_id': ..., 'checkpoint_id': ...}})`.
- Replaying from an old checkpoint forks a new branch — the original path stays stored too.
- `update_state(config, partial_update)` modifies state at a checkpoint and returns a new `checkpoint_id`.
- Always replay using the NEW `checkpoint_id` returned by `update_state()`, not the original one.

K. Key Takeaways

- **Persistence** is the ability to save and restore a LangGraph workflow's state over time, overriding the default behavior of erasing state at `END`.
- **Speciality**: persistence stores every intermediate state value, not just the final one — this is what enables fault tolerance and time travel.
- **Checkpointers**: the component that divides graph execution into checkpoints (one per superstep) and saves state to a database at each one.
- **Threads**: a unique `thread_id` isolates the checkpoints of one execution from another in the same database.
- **InMemorySaver** is a demo-only, RAM-based checkpointer; production systems use durable checkpointer (Postgres, Redis, etc.).
- Persistence unlocks four benefits: **short-term memory**, **fault tolerance**, **human-in-the-loop**, and **time travel**.
- **Fault tolerance**: `invoke(None, config)` resumes a crashed workflow from its last saved checkpoint instead of restarting.
- **Time travel**: replaying from a past `checkpoint_id` forks a new branch; `update_state()` can additionally modify state before replay, returning a new `checkpoint_id` to replay from.

R. Revision Sheet

Persistence

The ability to save and restore a workflow's state over time, instead of erasing it at END.

get_state vs get_state_history

`get_state` returns the final snapshot; `get_state_history` returns every stored snapshot for a thread.

Checkpoint

Divides graph execution into checkpoints (one per superstep) and saves state at each checkpoint to a database.

Fault Tolerance

Resuming a crashed workflow from its last checkpoint via `invoke(None, config)` instead of restarting from START.

Superstep → Checkpoint

Each superstep (a node, or a set of parallel nodes) of the graph corresponds to exactly one checkpoint.

Human-in-the-Loop

Execution is interrupted to wait for human input, then resumed from that exact point once input arrives.

thread_id

A unique identifier passed via config that isolates one execution's checkpoints from another's in the database.

Time Travel

Replaying execution from a past `checkpoint_id`, mainly for debugging; creates a new forked branch.

InMemorySaver

A RAM-based checkpointer for demos/learning; not used in production since data is lost on restart.

update_state()

Modifies state at a given checkpoint and returns a new `checkpoint_id` representing the updated snapshot.

I. Interview Questions

Q1. What is persistence in LangGraph, and what default behavior does it override?

Q2. Why does persistence store intermediate state values rather than only the final value? What capabilities does this enable?

Q3. What is a checkpoint, and how does it decide where to place checkpoints in a graph's execution?

Q4. Explain the relationship between a superstep and a checkpoint.

Q5. What problem does `thread_id` solve, and what happens if two executions share the same `thread_id`?

Q6. Why is `InMemorySaver` unsuitable for production, and what would you use instead?

Q7. How does LangGraph resume a workflow after a crash? What must be passed to `invoke()` to trigger a resume rather than a fresh run?

Q8. How is human-in-the-loop conceptually similar to fault tolerance, and how does it differ?

Q9. Describe how time travel works, including how a specific checkpoint is located and replayed.

Q10. What is the effect of calling `update_state()` at a checkpoint, and why must the newly returned `checkpoint_id` be used for the subsequent replay?

F. Further Reading

- [LangGraph official documentation — Persistence concept guide](#)
- [LangGraph official documentation — Checkpointer reference \(InMemorySaver, PostgresSaver, RedisSaver\)](#)
- [LangGraph official documentation — Time Travel guide](#)
- [LangGraph official documentation — Human-in-the-Loop guide](#)